

Arrays in C++ — DSA Notes

Part of my DSA placement prep journey. Previous: Time & Space Complexity · **Current: Arrays** · Next: Vectors

Table of Contents

1. What Is an Array?
2. Internal Working
3. Time & Space Complexity
4. Types of Arrays
5. Core Operations
6. When and Where to Use Arrays
7. Question Patterns
8. Practice Problems (Categorized)
9. Worked Example
10. Common Pitfalls
11. Quick Revision Summary

1. What Is an Array?

An **array** is a collection of elements of the same type, stored in **contiguous memory locations**, accessed using an index. It is the most fundamental linear data structure and the base on which vectors, strings, stacks, and hash tables are built.

```
int arr[5];                // static array, uninitialized
int arr2[5] = {1, 2, 3, 4, 5}; // static array, initialized
int arr3[] = {10, 20, 30};   // size inferred = 3
int* arr4 = new int[5];      // dynamic array (heap-allocated)
delete[] arr4;              // must manually free dynamic array
```

Key trait: since elements sit next to each other in memory, the address of `arr[i]` can be computed directly as `base_address + i * sizeof(type)` — this is *why* array access is $O(1)$.

2. Internal Working

2.1 Memory Layout

- All elements occupy a single contiguous block.
- Index `i` maps directly to a memory offset — no traversal needed to reach any element.

- This contiguity is also what gives arrays excellent **cache locality** (elements near each other are loaded into CPU cache together), making array traversal faster in practice than pointer-chasing structures like linked lists, even when both are technically $O(n)$.

2.2 Static vs Dynamic Arrays

	Static Array (<code>int arr[n]</code>)	Dynamic Array (<code>new int[n]</code>)
Memory	Stack (usually)	Heap
Size	Fixed at compile time	Fixed at allocation time, but decided at runtime
Resizing	Not possible	Not possible without manual realloc (copy to new block)
Cleanup	Automatic	Manual (<code>delete[]</code>) — leak risk if forgotten

This fixed-size limitation is exactly why `std::vector` exists — it wraps a dynamic array and automates the resize-and-copy process (see my Vector notes).

2.3 Multi-dimensional Arrays

```
int matrix[3][4];           // 3 rows, 4 cols – still contiguous memory, row-major order
```

Row-major order means `matrix[i][j]` is stored at `base + (i * cols + j) * sizeof(type)` — this is why row-wise traversal is cache-friendlier than column-wise traversal in a 2D array.

3. Time & Space Complexity

Operation	Time Complexity	Notes
Access <code>arr[i]</code>	$O(1)$	Direct offset calculation
Search (unsorted)	$O(n)$	Linear scan required
Search (sorted)	$O(\log n)$	Binary search applicable
Insertion at end (static array)	$O(1)$	Only if space pre-allocated
Insertion at beginning/middle	$O(n)$	Requires shifting all subsequent elements
Deletion at end	$O(1)$	—
Deletion at beginning/middle	$O(n)$	Requires shifting elements to fill gap
Traversal	$O(n)$	Visit every element once
Sorting	$O(n \log n)$	Comparison-based (merge/quick/heap sort)

Space complexity: $O(n)$ for n elements — no extra overhead per element (unlike linked lists which need extra pointer memory).

4. Types of Arrays

Type	Description	Typical Use
1D Array	Linear sequence of elements	Basic lists, single-attribute data
2D Array / Matrix	Grid of elements	Images, boards (chess, Sudoku), adjacency matrices
Static Array	Fixed size, stack-allocated	Small, known-size data, performance-critical embedded code
Dynamic Array	Runtime-sized, heap-allocated	Unknown size at compile time (usually replaced by <code>vector</code> in modern C++)
Jagged Array	Array of arrays with varying row lengths	Irregular data — e.g. adjacency lists in graphs

5. Core Operations

```

// Traversal
for (int i = 0; i < n; i++) cout << arr[i] << " ";

// Insertion at index (shift right)
for (int i = n; i > pos; i--) arr[i] = arr[i - 1];
arr[pos] = value;

// Deletion at index (shift left)
for (int i = pos; i < n - 1; i++) arr[i] = arr[i + 1];

// Reverse in-place (two-pointer)
int l = 0, r = n - 1;
while (l < r) { swap(arr[l], arr[r]); l++; r--; }

// Linear search
for (int i = 0; i < n; i++) if (arr[i] == target) return i;

// Binary search (sorted array only)
int lo = 0, hi = n - 1;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
}

```

6. When and Where to Use Arrays

6.1 In DSA / Interviews

- Whenever the **size is known and fixed**, and you need the fastest possible random access.
- As the underlying representation for strings, matrices, and hash table buckets.
- Building block for stacks and queues implemented via fixed-size buffers.
- Base case for almost every pattern: two pointers, sliding window, prefix sum, Kadane's, binary search.

6.2 In Real-World Engineering

- **Image processing:** a pixel grid is literally a 2D array (rows × cols × channels) — relevant to your ECE/signal-processing background.
- **Embedded/VLSI systems:** fixed-size buffers for sensor readings, register banks, or FIFO buffers where memory must be predictable and stack-allocated (no heap fragmentation allowed in constrained environments).
- **ML/feature vectors:** a fixed-length feature vector for a model (e.g., a 128-dim embedding) is naturally an array — contiguous memory matters for SIMD/vectorized operations in libraries like NumPy/Eigen.
- **Audio/signal buffers:** fixed-size circular buffers for streaming sensor or audio data.
- **Game development:** fixed-size grids (tile maps, chess boards) where bounds are known ahead of time.

6.3 When NOT to Use a Plain Array

- Size unknown or changes at runtime → use **vector**.
- Frequent insert/delete in the middle → consider **list** or a different structure.
- Need key-based lookup → use **map/unordered_map**.

7. Question Patterns

Pattern	Typical Ask	Example Problems
Two Pointers	Pair sum, palindrome, partitioning	Two Sum (sorted), Sort Colors, Container With Most Water
Sliding Window	Subarray/substring meeting a condition	Max Sum Subarray of Size K, Longest Substring Without Repeat
Prefix Sum	Range sum, subarray sum = k	Subarray Sum Equals K, Equilibrium Index
Kadane's Algorithm	Max/min contiguous subarray	Maximum Subarray, Maximum Product Subarray
Sorting-based	Dedup, closest pair, intervals	Merge Intervals, 3Sum
Binary Search	Search in sorted/rotated array	Search in Rotated Sorted Array, Find Peak Element
In-place Rearrangement	No extra space allowed	Move Zeroes, Rotate Array, Next Permutation
Matrix Traversal	Row/col/diagonal patterns	Spiral Matrix, Rotate Image, Set Matrix Zeroes
Frequency Counting	Majority element, duplicates	Majority Element, Find All Duplicates

8. Practice Problems (Categorized)

8.1 Basic Manipulation

1. Reverse an array in-place.
2. Find the maximum and minimum in a single pass.
3. Find the "missing number" in 1 to n.
4. Left rotate an array by d positions.
5. Find the frequency of each element (using a hash map).

8.2 Two Pointers

1. Two Sum (sorted array).
2. Sort an array of 0s, 1s, and 2s (Dutch National Flag).
3. Container With Most Water.
4. Trapping Rain Water.

8.3 Sliding Window

1. Maximum Sum Subarray of Size K.
2. Smallest Subarray with a given sum.
3. Longest Substring Without Repeating Characters.

8.4 Prefix Sum

1. Subarray Sum Equals K.
2. Equilibrium Index of an Array.
3. Range Sum Query (precomputed prefix sums).

8.5 Kadane's Algorithm

1. Maximum Subarray Sum.
2. Maximum Product Subarray.
3. Maximum Circular Subarray Sum.

8.6 Sorting-Based

1. Merge Intervals.
2. 3Sum.
3. Find the Duplicate Number.

8.7 Binary Search

1. Search in Rotated Sorted Array.
2. Find Peak Element.
3. Find First and Last Position of Element in Sorted Array.

8.8 In-place Rearrangement

1. Move Zeroes.
2. Rotate Array by K steps.
3. Next Permutation.

8.9 Matrix

1. Rotate Image (90° in-place).
2. Spiral Matrix Traversal.
3. Set Matrix Zeroes.

9. Worked Example

Problem: Kadane's Algorithm — Maximum Subarray Sum

Given an array of integers (may include negatives), find the contiguous subarray with the largest sum.

Approach - Track two variables: `currentSum` (best sum ending at current index) and `maxSum` (best sum seen so far).
- At each index, decide: extend the previous subarray, or start fresh from the current element — whichever gives a larger sum. - Update `maxSum` whenever `currentSum` exceeds it.

Code

```
int maxSubArray(vector<int>& nums) {
    int currentSum = nums[0];
    int maxSum = nums[0];
    for (int i = 1; i < nums.size(); i++) {
        currentSum = max(nums[i], currentSum + nums[i]);
        maxSum = max(maxSum, currentSum);
    }
    return maxSum;
}
```

Complexity - Time: $O(n)$ — single pass. - Space: $O(1)$ — only two variables tracked.

Exam-answer structure (5-mark format) 1. Restate problem + constraints (1 mark) 2. State approach/intuition (1 mark) 3. Write clean, compilable code (2 marks) 4. State time & space complexity explicitly (1 mark)

10. Common Pitfalls

Pitfall	Why It Matters
Off-by-one errors in loop bounds	$i \leq n$ vs $i < n$ is the single most common source of bugs on array problems
Accessing <code>arr[n]</code> on an n -sized array	Undefined behavior — index range is 0 to $n-1$
Forgetting <code>delete[]</code> on dynamic arrays	Memory leak — always pair <code>new[]</code> with <code>delete[]</code>
Confusing row-major traversal with column-major	Column-wise traversal on a large 2D array is cache-unfriendly and slower in practice
Modifying array size assumptions mid-problem	Plain arrays cannot resize — attempting to "add" beyond declared size overflows into adjacent memory
Using linear search when the array is already sorted	Wastes an opportunity for $O(\log n)$ binary search

11. Quick Revision Summary

- Array = contiguous memory, $O(1)$ random access, fixed size.
- Insertion/deletion at the end is $O(1)$; at the beginning/middle is $O(n)$ due to shifting.
- Row-major storage means row-wise traversal beats column-wise for cache performance.
- Core patterns to have muscle memory for: two pointers, sliding window, prefix sum, Kadane's, binary search.
- Plain arrays are the right choice when size is fixed and known ahead of time; otherwise, use `vector`.